

Home4U

AI-Assisted, Budget-Aware Interior Style Matching Platform

Team Number: 4

Team Members & Roles:

- Caleb Ponce — Team Lead / System Architecture
- Tyler Morris — Backend & AI Integration
- Christopher Quach — Frontend Development
- Mason Lee — Data Modeling & Scoring Engine
- Dias Almat — Technical Writer

Team Lead Email: cponce8@sfsu.edu
Date: February 8, 2026

Revision History

Version	Date	Author(s)	Description
0.1	___	Team	Initial draft created
0.2	___	Team	Added requirements and analysis
1.0	___	Caleb Ponce	Final review and submission

Table of Contents

Home4U.....	1
Table of Contents.....	1
1. Executive Summary.....	2
2. Main Use Cases.....	3
2.1 Actors.....	3

2.2 Primary Use Cases — User.....	3
2.3 Primary Use Cases — Administrator.....	4
3. Main Data Items and Entities.....	4
4. Functional Requirements.....	5
4.1 User Requirements.....	5
4.2 Administrator Requirements.....	5
5. Non-Functional Requirements.....	5
5.1 Performance.....	6
5.2 Reliability.....	6
5.3 Security.....	6
5.4 Usability.....	6
5.5 Scalability.....	6
5.6 Maintainability.....	6
6. Competitive Analysis.....	6
Unique Features of Home4U.....	7
7. Project Readiness Checklist.....	7
8. High Level Architecture and Technologies.....	7
8.1 Architecture Overview.....	7
8.2 Approved Technologies.....	8
9. Team Contributions.....	8

1. Executive Summary

Home4U is a web-based platform designed to help renters and first-time apartment dwellers transform their rooms into desired aesthetic styles using structured, data-driven guidance. While existing inspiration platforms such as Pinterest and Houzz provide visual references, they do not translate those references into actionable steps tailored to a user’s actual room and budget. Home4U bridges that gap.

Users upload a photo of their room, select a room type (living room, bedroom, kitchen, or bathroom), and choose one or more target styles such as Modern, Japandi, or Industrial. The system uses AI-assisted image analysis to suggest descriptive room tags (e.g., furniture types, materials, color palette), which the user confirms or edits. Based on confirmed tags, the platform computes a resemblance score using a weighted style-matching algorithm.

Home4U then generates prioritized recommendations to help close the style gap. Users may input a budget constraint, allowing the system to rank suggested improvements by

impact-to-cost ratio. The platform also provides curated product suggestions aligned with the selected style and room type.

Unlike generic inspiration tools or manual shopping searches, Home4U offers explainable style scoring, budget-aware prioritization, and structured recommendations. While initially focused on renters, the architecture supports future expansion into broader residential markets.

2. Main Use Cases

2.1 Actors

User (Renter)

Administrator

2.2 Primary Use Cases — User

UC-01 Create and Manage Room Projects

User creates a room project representing a specific physical room and may edit or delete it.

UC-02 Analyze a Room to Extract Descriptive Attributes

User submits a room image; the system generates suggested descriptive attributes.

UC-03 Confirm and Customize Room Attributes

User reviews, confirms, edits, or removes AI-suggested attributes.

UC-04 Discover Best-Fit Interior Styles

User requests ranked style matches based on confirmed room attributes.

UC-05 Evaluate Style Gap for a Selected Style

The user selects a target style and views resemblance score and missing attributes.

UC-06 Generate Budget-Constrained Improvement Plan

User inputs budget; system ranks improvements by impact-to-cost ratio.

UC-07 Explore Product-Level Recommendations

User views suggested items aligned with improvement categories.

UC-08 Preview Placement of Recommended Items

The user interactively places recommended items onto the room image.

UC-09 Save and Compare Design Iterations

The user saves style configurations and compares multiple design attempts.

2.3 Primary Use Cases — Administrator

1. Manage style definitions
 2. Assign style tag weights
 3. Manage curated product catalog
 4. Maintain system configuration
-

3. Main Data Items and Entities

User

Stores authentication credentials and profile information.

RoomProject

Represents a user-created room instance including room type and metadata.

RoomImage

Stores uploaded image file reference and associated metadata.

Style

Represents predefined interior design styles.

StyleBoardImage

Curated reference images illustrating each style.

Tag

Descriptive labels used to categorize room elements.

RoomTag

Associates suggested and confirmed tags with a room project.

StyleTag

Defines required and weighted tags for each style.

ResemblanceScore

Stores computed similarity score between a room and selected style.

Recommendation

Actionable suggestions generated by the system.

ProductItem

Curated product suggestion aligned with style and category.

PlacedItem

Stores preview placement metadata (x, y coordinates, scale, rotation, product reference).

LayoutSnapshot

Stores saved preview configurations for comparison.

ProductTemplate

Defines product search parameters and estimated price range for each recommendation category.

4. Functional Requirements

4.1 User Requirements

USR-01: Users shall create accounts and authenticate securely.

USR-02: Users shall create and manage room projects.

USR-03: Users shall upload room images for analysis.

USR-04: Users shall select one or multiple target styles.

USR-05: The system shall generate AI-assisted tag suggestions from uploaded images.

USR-06: Users shall confirm, edit, or remove suggested tags.

USR-07: The system shall compute resemblance scores using weighted style matching.

USR-08: Users shall compare resemblance scores across multiple styles.

USR-09: Users shall input a budget constraint.

USR-10: The system shall prioritize recommendations based on impact and budget.

USR-11: The system shall display curated product suggestions.

USR-12: Users shall save and revisit prior room projects.

4.2 Administrator Requirements

ADM-01: Administrators shall manage style definitions.

ADM-02: Administrators shall assign and modify style tag weights.

ADM-03: Administrators shall manage curated product items.
ADM-04: Administrators shall maintain system configuration integrity.

5. Non-Functional Requirements

5.1 Performance

NFR-01: Resemblance scoring shall complete within 5 seconds under normal load.

5.2 Reliability

NFR-02: System shall maintain 99% uptime during evaluation period.

5.3 Security

NFR-03: Passwords shall be hashed and salted.
NFR-04: JWT authentication shall be implemented.
NFR-05: Uploaded images shall be access-controlled.

5.4 Usability

NFR-06: Interface shall be responsive for desktop and mobile devices.
NFR-07: Workflow shall require minimal user steps.

5.5 Scalability

NFR-08: Database schema shall support multiple room projects per user.

5.6 Maintainability and Coding Standards

NFR-09: Backend code shall conform to PEP 8 standards.
NFR-10: Code formatting shall be enforced using Black.
NFR-11: Linting shall be enforced using Ruff or Flake8.
NFR-12: Core backend modules shall use Python type hints.
NFR-13: Frontend code shall follow ESLint + Prettier formatting rules.
NFR-14: API endpoints shall follow REST naming conventions and consistent JSON response

schemas.

NFR-15: GitHub pull requests shall require code review prior to merge.

6. Competitive Analysis

Product	Photo Personalization	Style Scoring	Budget Filter	Shopping Integration	Limitation
Pinterest https://www.pinterest.com/	No	No	No	Indirect	Inspiration only
Houzz https://www.houzz.com/	Limited	No	No	Yes	No scoring
IKEA Planner https://www.ikea.com/planner	Limited	No	Limited	Yes	Store-specific
Havenly havenly	Yes	No	No	Yes	Paid service
Wayfair Ideas wayfair	Limited	No	No	Yes	No personalization engine

Unique Features of Home4U

1. Human-in-the-loop AI tagging
2. Weighted resemblance scoring system
3. Multi-style comparison engine

4. Budget-aware prioritization algorithm
 5. Room-type-specific recommendations
-

7. Project Readiness Checklist

- ☒ Meeting schedule established
 - ☒ Roles assigned
 - ☒ Tools selected and understood
 - ☒ Repository organized correctly
-

8. High Level Architecture and Technologies

8.1 Architecture Overview

Frontend (React)

- REST API (FastAPI)
- PostgreSQL Database
- OpenAI Vision API (tag suggestion)
- Nginx Reverse Proxy
- AWS EC2 (Ubuntu 22.04 LTS)

→ HTTPS via Nginx

→ TLS certificate via Let's Encrypt (Certbot)

8.2 Approved Technologies

Cloud: AWS EC2 Free Tier

Operating System: Ubuntu 22.04 LTS

Web Server: Nginx

Backend: Python 3.10 + FastAPI

Database: PostgreSQL 16

Frontend: React 18 + Vite

SSL/TLS: Let's Encrypt (Certbot)
AI Model: OpenAI Vision API
Development Environment: Visual Studio Code

9. Team Contributions

Member	Role	Milestone 1 Contribution
Caleb Ponce	Team Lead	Architecture planning, executive summary, final review
Tyler Morris	Backend	API planning and data modeling
Christopher Quach	Frontend	Use case documentation and UI flow
Mason Lee	Data	Functional and non-functional requirements
Dias Almat	Writer	Formatting, TOC, and PDF preparation

The system separates AI-assisted feature extraction from deterministic style scoring to ensure explainability, transparency, and architectural control.